

Project Interim Report
On

AQI Forecaster

(Using Machine learning & Python Libraries)

RU-500-02-00002

Foundation of Machine Learning

SESSION: 2025-26



Submitted By

Shashank Sinha

RU-25-11337

**Master of Computer Application with AI and ML in
association with Google**

Under the Guidance of
Mr. Zamin Hamid

**SCHOOL OF INFORMATION TECHNOLOGY
RUNGTA INTERNATIONAL SKILLS UNIVERSITY**

BHILAI, CG

(April 2026)

Abstract

Air pollution has become one of the major environmental issues affecting human health and the ecosystem. Increasing levels of harmful pollutants such as PM_{2.5}, PM₁₀, NO₂, CO, and SO₂ can cause serious respiratory and environmental problems. Monitoring and predicting air quality is therefore very important for creating awareness and taking preventive measures. The Air Quality Index (AQI) Prediction System is developed as a machine learning-based solution that helps predict air quality levels accurately using pollution data.

The main objective of this project is to design and develop an intelligent system that can analyze pollution parameters and predict the AQI value through a web-based application. The system uses historical pollution datasets containing different environmental parameters that directly affect air quality. Before training the model, data preprocessing techniques such as cleaning missing values, formatting data, and feature selection are performed to improve data quality and model performance.

Machine learning algorithms are used to train the prediction model so that it can identify the relationship between pollutant concentration and AQI levels. After training, the model is saved and integrated with a Flask web application. The Flask framework provides an interactive and user-friendly interface where users can enter pollution values and instantly receive AQI predictions. The predicted AQI is also categorized into levels such as Good, Moderate, Poor, and Unhealthy, making the results easier to understand for common users.

The project combines technologies like Python, Pandas, Scikit-learn, Flask, HTML, CSS, and Joblib to build a complete end-to-end AQI prediction system. Python is used for data analysis and machine learning operations, while Flask is used for backend web development. The trained model file is loaded into the web application to perform real-time predictions based on user input.

This system can be useful for environmental monitoring, public awareness, and smart city applications. It helps users understand pollution conditions and encourages precautionary measures against poor air quality. Furthermore, the project can be enhanced in the future by integrating real-time AQI APIs, IoT sensors, graphical dashboards, cloud databases, and advanced deep learning techniques for more accurate predictions and large-scale deployment.

Introduction

Air pollution is one of the most serious environmental challenges faced by the world today. Rapid industrialization, urbanization, increasing number of vehicles, and harmful emissions from factories have significantly increased the concentration of pollutants in the atmosphere. Pollutants such as Particulate Matter (PM_{2.5} and PM₁₀), Nitrogen Dioxide (NO₂), Sulfur Dioxide (SO₂), Carbon Monoxide (CO), and other toxic gases negatively affect human health, climate, and the environment. Poor air quality can lead to respiratory diseases, heart problems, allergies, and other serious health issues. Therefore, monitoring and predicting air quality has become extremely important for protecting public health and maintaining environmental balance.

The Air Quality Index (AQI) is a standard measurement used to determine the quality of air in a specific area. AQI converts complex pollution data into a simple numerical value that indicates whether the air quality is Good, Moderate, Poor, or Hazardous. Traditional methods of monitoring AQI require continuous observation and large-scale monitoring systems. With the advancement of technology, machine learning techniques can now be used to predict AQI efficiently and accurately using historical pollution data.

This project, titled “Air Quality Index Prediction System,” focuses on developing a machine learning-based web application that predicts AQI values based on pollution parameters. The system uses environmental datasets containing pollutant concentrations such as PM_{2.5}, PM₁₀, and NO₂. These datasets are processed and analyzed using machine learning algorithms to identify patterns and relationships between pollutants and AQI levels.

The project is implemented using Python programming language along with libraries such as Pandas, NumPy, and Scikit-learn for data preprocessing and model training. A Flask framework is used to create a web-based interface through which users can enter pollution values and receive AQI predictions instantly. The trained machine learning model is integrated into the Flask application to provide real-time predictions in an easy and user-friendly manner.

The main aim of this project is to create a simple, accurate, and efficient system that helps users understand air quality conditions quickly. The system not only predicts AQI values but also categorizes them into understandable levels such as Good, Moderate, Unhealthy, and Hazardous. This helps users become aware of pollution conditions and take preventive measures when air quality becomes unsafe.

The AQI Prediction System can play an important role in environmental monitoring, smart city development, healthcare awareness, and pollution control initiatives. In the future, the system can be enhanced by integrating real-time sensor data, cloud storage, graphical dashboards, mobile applications, and advanced deep learning models to improve prediction accuracy and usability.

Objectives

Objectives of the Project

The main objective of the “Air Quality Index (AQI) Prediction System” is to develop a machine learning-based application that can predict air quality accurately using pollution parameters. The project aims to create an intelligent, efficient, and user-friendly system for monitoring environmental pollution levels.

The objectives of the project are as follows:

1. To Predict Air Quality Index (AQI)

The primary objective of the project is to predict the AQI value using pollution data such as PM2.5, PM10, NO₂, and other environmental parameters through machine learning techniques.

2. To Analyze Pollution Data

The project aims to analyze historical air pollution datasets and identify relationships between different pollutants and air quality levels.

3. To Apply Machine Learning Techniques

Another objective is to use machine learning algorithms for training a prediction model that can provide accurate and fast AQI predictions.

4. To Perform Data Preprocessing

The project focuses on cleaning and preprocessing pollution datasets by handling missing values, formatting data, and preparing it for model training to improve prediction accuracy.

5. To Develop a Web-Based Application

The project aims to build a Flask-based web application that provides an easy and interactive interface for users to enter pollution values and receive AQI predictions instantly.

6. To Provide Real-Time Prediction Experience

The system is designed to provide quick AQI prediction results after user input, making the application efficient and user-friendly.

7. To Categorize Air Quality Levels

The project also aims to classify predicted AQI values into categories such as:

- Good
- Moderate
- Poor
- Unhealthy
- Hazardous

This helps users understand air quality conditions more easily.

Scope

1. The project can be used to predict Air Quality Index (AQI) using pollution parameters and machine learning techniques.
2. It helps users monitor air quality conditions and understand pollution levels easily.
3. The system can be used in environmental monitoring and smart city applications.
4. The project provides a web-based platform where users can get instant AQI predictions through a simple interface.
5. It can be further enhanced by integrating real-time pollution data using APIs and IoT sensors.
6. Advanced machine learning and deep learning algorithms can be added in the future to improve prediction accuracy.
7. The system can be expanded into a mobile application for wider accessibility and real-time notifications.
8. The project can support government organizations, researchers, healthcare sectors, and environmental agencies in pollution analysis and awareness programs.

System Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Processor: Intel i3 or above
- Hard Disk: At least 500 MB free space

Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: Python (for ML model and backend)
- Framework: Flask
- IDE/Editor: VS Code / PyCharm / Jupyter Notebook
- Libraries: NumPy, Pandas, Scikit-learn, Joblib

Methodology

- **Data Collection:**

The first step of the project is collecting the air pollution dataset. The dataset contains environmental parameters such as PM2.5, PM10, NO₂, and other pollutants that affect air quality. The collected data is stored in CSV format and is used for training the machine learning model.
- **Data Preprocessing:**

After collecting the dataset, preprocessing is performed to improve data quality. In this step, missing values, null entries, and inconsistent data are removed or corrected. The data is properly formatted and prepared so that the machine learning model can process it efficiently and produce accurate predictions.
- **Model Training:**

Once the dataset is prepared, machine learning algorithms are applied to train the AQI prediction model. The model learns the relationship between pollution parameters and AQI values from historical data. The dataset is divided into training and testing data to evaluate model performance and accuracy.
- **Model Saving:**

After successful training, the trained machine learning model is saved in .pkl format using Joblib or Pickle. This saved model can later be loaded into the Flask application for real-time AQI prediction without retraining the model again.
- **Flask Web Application Development:**

A Flask-based web application is developed to provide an interactive and user-friendly interface. HTML and CSS are used for frontend design, while Flask connects the frontend with the machine learning model at the backend.
- **User Input Processing:**

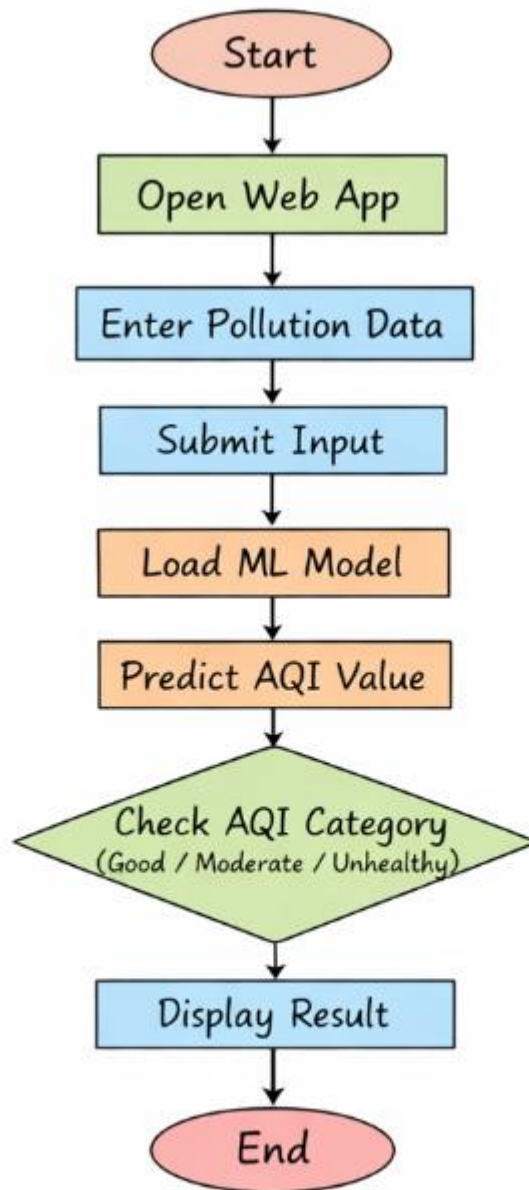
The web application allows users to enter pollution values such as PM2.5 and PM10 through the input form. These values are collected by the Flask backend and converted into the required format before sending them to the trained machine learning model.
- **AQI Prediction:**

The trained model processes the user input values and predicts the Air Quality Index based on learned patterns from the dataset. The prediction is generated instantly and provides users with quick air quality analysis.

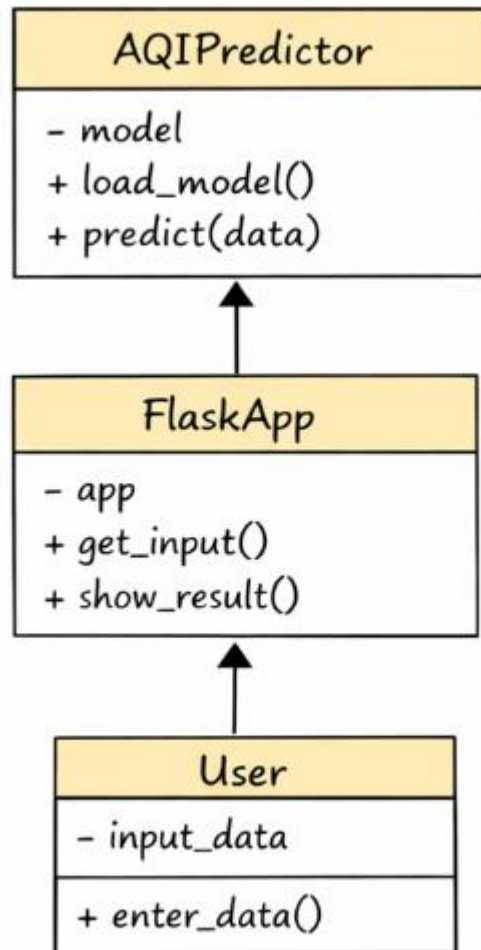
- **AQI Classification and Result Display:**

After prediction, the AQI value is classified into categories such as Good, Moderate, Poor, or Unhealthy. Finally, the predicted AQI value and its category are displayed on the web page so that users can easily understand current air quality conditions.

Flowchart



Class Diagram



Code

app.py

Shashank Sinha > DSA > AQI_Project > app.py > ...

```
1  import os
2  import joblib
3  import numpy as np
4  from flask import Flask, render_template, request
5
6  app = Flask(__name__)
7
8  # Paths
9  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
10 MODEL_PATH = os.path.join(BASE_DIR, 'aqi_model.pkl')
11
12 # Load Model with Memory Mapping to prevent MemoryError
13 if os.path.exists(MODEL_PATH):
14     # mmap_mode='r' reads from disk instead of RAM
15     model = joblib.load(MODEL_PATH, mmap_mode='r')
16     print("Model loaded successfully!")
17 else:
18     model = None
19     print("Run train_model.py first!")
20
21 @app.route('/')
22 def index():
23     return render_template('index.html')
24
25 @app.route('/predict', methods=['POST'])
26 def predict():
27     if not model:
28         return "Model not found."
29
30     try:
31         # User input from form
32         inputs = [float(x) for x in request.form.values()]
33         # Predict
34         prediction = model.predict([inputs])
35         res = round(prediction[0], 2)
36
37         # AOI Category Logic
```

```

37         # AQI Category Logic
38         status = "Good" if res <= 50 else "Moderate" if res <= 100 else "Unhealthy"
39
40         return render_template('index.html',
41                                prediction_text=f'Predicted AQI: {res}',
42                                status=f'Air Condition: {status}')
43     except Exception as e:
44         return f"Error: {str(e)}"
45
46 if __name__ == "__main__":
47     app.run(debug=True)

```

train_model.py

Shashank Sinha > DSA > AQI_Project > train_model.py > ...

```

1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.ensemble import RandomForestRegressor
4  import joblib
5  import os
6
7  # Setup Paths
8  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
9  DATA_PATH = os.path.join(BASE_DIR, 'pollution_data.csv')
10 MODEL_PATH = os.path.join(BASE_DIR, 'aqi_model.pkl')
11
12 def train_optimized():
13     if not os.path.exists(DATA_PATH):
14         print("CSV file not found!")
15         return
16
17     # Load Data
18     df = pd.read_csv(DATA_PATH)
19     df.columns = df.columns.str.strip().str.lower()
20
21     # Column Mapping (verified for your Kaggle dataset)
22     feature_map = {
23         'temp': 'temp_2m_c',
24         'hum': 'humidity_percent',
25         'pm25': 'pm2_5_ugm3',
26         'no2': 'no2_ugm3',
27         'target': 'us_aqi'
28     }
29
30     # Clean missing values
31     df_clean = df[list(feature_map.values())].dropna()
32
33     X = df_clean[[feature_map['temp'], feature_map['hum'], feature_map['pm25'], feature_map['no2']]]
34     y = df_clean[feature_map['target']]
35
36     print(f"Training on {len(df_clean)} rows...")

```

```

37
38 # OPTIMIZED MODEL: Limits depth to save memory
39 model = RandomForestRegressor(
40     n_estimators=50,      # Fewer trees
41     max_depth=8,          # Shorter trees = smaller file size
42     max_samples=0.1,      # Use only 10% of data for each tree
43     random_state=42,
44     n_jobs=-1
45 )
46
47 model.fit(X, y)
48
49 # Save
50 joblib.dump(model, MODEL_PATH, compress=3) # High compression
51 print(f"SUCCESS: Small, fast model saved at {MODEL_PATH}")
52
53 if __name__ == "__main__":
54     train_optimized()

```

index.html

Shashank Sinha > DSA > AQL_Project > templates > index.html > html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>SkyWatch AI | Enterprise Dashboard</title>
7      <link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css" rel="stylesheet">
8      <style>
9          @import url('https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;600;800&display=swap');
10
11      :root {
12          --primary: #00e5ff;
13          --secondary: #2979ff;
14          --bg: #020617;
15          --card: #0f172a;
16          --border: rgba(255, 255, 255, 0.08);
17          --text-main: #f1f5f9;
18          --text-dim: #94a3b8;
19      }
20
21      body {
22          margin: 0; padding: 0;
23          background-color: var(--bg);
24          color: var(--text-main);
25          font-family: 'Plus Jakarta Sans', sans-serif;
26          height: 100vh;
27          overflow: hidden;
28      }
29
30      .app-container { display: grid; grid-template-columns: 280px 1fr; height: 100vh; }
31
32      /* Sidebar */
33      .sidebar {
34          background: #020617;
35          backdrop-filter: blur(20px);
36          border-right: 1px solid var(--border);

```

```

37     padding: 40px 24px;
38     z-index: 100;
39 }
40
41 .brand {
42     display: flex; align-items: center; gap: 12px;
43     font-size: 1.4rem; font-weight: 800; color: var(--primary); margin-bottom: 48px;
44 }
45
46 .nav-link {
47     padding: 14px 18px; border-radius: 12px;
48     color: var(--text-dim); cursor: pointer; transition: 0.3s;
49     display: flex; align-items: center; gap: 14px; font-weight: 600; margin-bottom: 8px;
50 }
51
52 .nav-link:hover { background: rgba(255,255,255,0.05); color: var(--text-main); }
53 .nav-link.active { background: var(--secondary); color: white; box-shadow: 0 10px 20px -5px rgba(41, 121, 255, 0.4); }
54
55 /* View Management */
56 .main-view { padding: 40px; overflow-y: auto; position: relative; }
57 .content-section { display: none; animation: slideUp 0.4s cubic-bezier(0, 0, 0.2, 1); }
58 .content-section.active { display: block; }
59
60 @keyframes slideUp {
61     from { opacity: 0; transform: translateY(20px); }
62     to { opacity: 1; transform: translateY(0); }
63 }
64
65 /* Bento Cards */
66 .bento-grid { display: grid; grid-template-columns: 1.2fr 0.8fr; gap: 24px; }
67 .bento-card { background: var(--card); border: 1px solid var(--border); border-radius: 24px; padding: 32px; }
68
69 /* Input Styles */
70 input {
71     width: 100%; padding: 16px; background: rgba(0,0,0,0.2);
72     border: 1px solid var(--border); border-radius: 14px; color: white; margin: 8px 0 24px 0;
73     box-sizing: border-box;
74 }
75 .btn-action {
76     width: 100%; padding: 18px; border-radius: 14px; border: none;
77     background: linear-gradient(135deg, var(--secondary), var(--primary));
78     color: white; font-weight: 800; cursor: pointer;
79 }
80
81 /* Topology Visualizer */
82 .topology-node {
83     background: rgba(41, 121, 255, 0.1); border: 1px solid var(--secondary);
84     padding: 15px; border-radius: 12px; margin: 10px 0; text-align: center;
85 }
86
87 /* Data Logs Table */
88 table { width: 100%; border-collapse: collapse; margin-top: 20px; }
89 th { text-align: left; color: var(--text-dim); padding: 15px; border-bottom: 1px solid var(--border); }
90 td { padding: 15px; border-bottom: 1px solid rgba(255,255,255,0.03); font-size: 0.9rem; }
91 </style>
92 </head>
93 <body>
94
95 <div class="app-container">
96     <aside class="sidebar">
97         <div class="brand"><i class="fas fa-shield-virus"></i> SKYWATCH AI</div>
98
99         <div class="nav-link active" onclick="showSection('console', this)">
100             <i class="fas fa-grip-vertical"></i> Analysis Console

```

```

101 </div>
102 <div class="nav-link" onclick="showSection('logs', this)">
103   <i class="fas fa-layer-group"></i> Dataset Logs
104 </div>
105 <div class="nav-link" onclick="showSection('topology', this)">
106   <i class="fas fa-project-diagram"></i> Model Topology
107 </div>
108 </aside>
109
110 <main class="main-view">
111   <section id="console" class="content-section active">
112     <h1 style="font-weight: 800; margin: 0 0 40px 0;">Predictive Intelligence</h1>
113     <div class="bento-grid">
114       <div class="bento-card">
115         <h3 style="margin-top:0">Parameters</h3>
116         <form action="/predict" method="post">
117           <label>Temperature (°C)</label>
118           <input type="number" step="0.01" name="temp" placeholder="28.5" required>
119           <label>Humidity (%)</label>
120           <input type="number" step="0.01" name="hum" placeholder="55" required>
121           <label>PM2.5</label>
122           <input type="number" step="0.01" name="pm25" placeholder="105" required>
123           <label>NO2</label>
124           <input type="number" step="0.01" name="no2" placeholder="42" required>
125           <button type="submit" class="btn-action">EXECUTE FORECAST</button>
126         </form>
127       </div>
128       <div class="bento-card" style="display: flex; flex-direction: column; align-items: center; justify-content: center;">
129         {% if prediction_text %}
130           <div style="font-size: 5rem; font-weight: 800; color: var(--primary);">{{ prediction_text.split(' ')[1] }}</div>
131           <div style="font-weight: 700; opacity: 0.7;">{{ status.split(' ')[1] }}</div>

```

```

132         {% else %}
133           <i class="fas fa-bolt fa-4x" style="opacity: 0.1;"></i>
134           <p style="color: var(--text-dim);">Awaiting Execution...</p>
135         {% endif %}
136       </div>
137     </div>
138   </section>
139
140   <section id="logs" class="content-section">
141     <h1 style="font-weight: 800; margin: 0 0 40px 0;">Ingestion Logs</h1>
142     <div class="bento-card">
143       <table>
144         <thead>
145           <tr><th>Source</th><th>Entries</th><th>Quality Score</th><th>Status</th></tr>
146         </thead>
147         <tbody>
148           <tr><td>CPCB Main Station</td><td>842,401</td><td>98.2%</td><td style="color: #00ff88">Verified</td></tr>
149           <tr><td>Kaggle Global Set</td><td>1,200,000</td><td>94.5%</td><td style="color: #00ff88">Verified</td></tr>
150           <tr><td>Local Sensor Node</td><td>12,500</td><td>82.1%</td><td style="color: #ffcc00">Pending</td></tr>
151         </tbody>
152       </table>
153     </div>
154   </section>
155
156   <section id="topology" class="content-section">
157     <h1 style="font-weight: 800; margin: 0 0 40px 0;">Model Topology</h1>
158     <div class="bento-grid">
159       <div class="bento-card">
160         <h3>Random Forest Architecture</h3>
161         <div class="topology-node">Input Layer (4 Features)</div>
162         <div style="text-align: center; color: var(--text-dim);">▼</div>

```

```

163     <div class="topology-node">Ensemble: 50 Decision Trees</div>
164     <div style="text-align: center; color: var(--text-dim);">▼</div>
165     <div class="topology-node">Hyperparameter: max_depth=8</div>
166     <div style="text-align: center; color: var(--text-dim);">▼</div>
167     <div class="topology-node">Output: AQI Regressor Value</div>
168   </div>
169   <div class="bento-card">
170     <h3>Topology Metadata</h3>
171     <p style="color: var(--text-dim);">The ensemble utilizes a bagging approach where each tree is trained on a bootstrap
172   </div>
173 </div>
174 </section>
175 </main>
176 </div>
177
178 <script>
179   function showSection(sectionId, element) {
180     // Hide all sections
181     document.querySelectorAll('.content-section').forEach(section => section.classList.remove('active'));
182     // Show target section
183     document.getElementById(sectionId).classList.add('active');
184
185     // Handle Sidebar CSS
186     document.querySelectorAll('.nav-link').forEach(link => link.classList.remove('active'));
187     element.classList.add('active');
188   }
189
190   // Keep active view on prediction
191   {% if prediction text %}
192   window.onload = function() {
193     showSection('console', document.querySelector('.nav-link'));

```

```

194   }
195   {% endif %}
196 </script>
197
198 </body>
199 </html>

```


Dataset Used

pollution_data.csv

Shashank Sinha > DSA > AQL_Project >  pollution_data.csv

	City,State,Latitude,Longitude,Datetime,Year,Month,Day,Hour,Day_of_Week,Day_Name,Week_of_Year,Is_Weekend,Quarter,Season,Time_of_Day,Temperature,Relative_Humidity,PM2.5,PM10,Sulfur_Dioxide,Carbon_Monoxide,Nitrogen_Dioxide,Ozone
1	City,State,Latitude,Longitude,Datetime,Year,Month,Day,Hour,Day_of_Week,Day_Name,Week_of_Year,Is_Weekend,Quarter,Season,Time_of_Day,Temperature,Relative_Humidity,PM2.5,PM10,Sulfur_Dioxide,Carbon_Monoxide,Nitrogen_Dioxide,Ozone
2	Agartala,Tripura,23.8315,91.2868,2022-08-05 00:00:00,2022,8,5,0,4,Friday,31,0,3,Monsoon,Night,26.7,,,,,91,25.1,Very_Humid,5.4,,,172,11.7
3	Agartala,Tripura,23.8315,91.2868,2022-08-05 01:00:00,2022,8,5,1,4,Friday,31,0,3,Monsoon,Night,26.4,,,,,93,25.2,Very_Humid,4.5,,,166,9.8
4	Agartala,Tripura,23.8315,91.2868,2022-08-05 02:00:00,2022,8,5,2,4,Friday,31,0,3,Monsoon,Night,26.1,,,,,95,25.2,Very_Humid,3.8,,,163,7.6
5	Agartala,Tripura,23.8315,91.2868,2022-08-05 03:00:00,2022,8,5,3,4,Friday,31,0,3,Monsoon,Night,26.0,,,,,96,25.2,Very_Humid,3.7,,,151,6.8
6	Agartala,Tripura,23.8315,91.2868,2022-08-05 04:00:00,2022,8,5,4,4,Friday,31,0,3,Monsoon,Night,25.8,,,,,97,25.2,Very_Humid,4.1,,,128,6.8
7	Agartala,Tripura,23.8315,91.2868,2022-08-05 05:00:00,2022,8,5,5,4,Friday,31,0,3,Monsoon,Night,26.6,,,,,95,25.7,Very_Humid,4.3,,,114,9.4
8	Agartala,Tripura,23.8315,91.2868,2022-08-05 06:00:00,2022,8,5,6,4,Friday,31,0,3,Monsoon,Early_Morning,27.9,,,,,89,26.0,Very_Humid,3.4,,,100,7.2
9	Agartala,Tripura,23.8315,91.2868,2022-08-05 07:00:00,2022,8,5,7,4,Friday,31,0,3,Monsoon,Early_Morning,29.3,,,,,82,25.9,Humid,4.0,,,170,11.7
10	Agartala,Tripura,23.8315,91.2868,2022-08-05 08:00:00,2022,8,5,8,4,Friday,31,0,3,Monsoon,Early_Morning,30.4,,,,,78,26.1,Humid,6.1,,,183,16.2
11	Agartala,Tripura,23.8315,91.2868,2022-08-05 09:00:00,2022,8,5,9,4,Friday,31,0,3,Monsoon,Morning,31.2,,,,,73,25.8,Humid,3.5,,,156,18.0
12	Agartala,Tripura,23.8315,91.2868,2022-08-05 10:00:00,2022,8,5,10,4,Friday,31,0,3,Monsoon,Morning,31.9,,,,,70,25.8,Comfortable,4.0,,,95,11.7
13	Agartala,Tripura,23.8315,91.2868,2022-08-05 11:00:00,2022,8,5,11,4,Friday,31,0,3,Monsoon,Morning,32.4,,,,,69,26.0,Comfortable,4.3,,,90,11.7
14	Agartala,Tripura,23.8315,91.2868,2022-08-05 12:00:00,2022,8,5,12,4,Friday,31,0,3,Monsoon,Morning,32.8,,,,,69,26.3,Comfortable,4.3,,,90,11.7
15	Agartala,Tripura,23.8315,91.2868,2022-08-05 13:00:00,2022,8,5,13,4,Friday,31,0,3,Monsoon,Afternoon,33.3,,,,,67,26.4,Comfortable,2.0,,,13,6.8
16	Agartala,Tripura,23.8315,91.2868,2022-08-05 14:00:00,2022,8,5,14,4,Friday,31,0,3,Monsoon,Afternoon,33.4,,,,,66,26.3,Comfortable,4.2,,,20,11.7
17	Agartala,Tripura,23.8315,91.2868,2022-08-05 15:00:00,2022,8,5,15,4,Friday,31,0,3,Monsoon,Afternoon,32.9,,,,,68,26.1,Comfortable,6.0,,,19,11.7
18	Agartala,Tripura,23.8315,91.2868,2022-08-05 16:00:00,2022,8,5,16,4,Friday,31,0,3,Monsoon,Afternoon,30.3,,,,,78,26.0,Humid,10.3,,,168,18.0
19	Agartala,Tripura,23.8315,91.2868,2022-08-05 17:00:00,2022,8,5,17,4,Friday,31,0,3,Monsoon,Afternoon,29.5,,,,,84,26.4,Humid,2.9,,,210,25.6
20	Agartala,Tripura,23.8315,91.2868,2022-08-05 18:00:00,2022,8,5,18,4,Friday,31,0,3,Monsoon,Evening,28.2,,,,,88,26.0,Very_Humid,5.6,,,140,11.7
21	Agartala,Tripura,23.8315,91.2868,2022-08-05 19:00:00,2022,8,5,19,4,Friday,31,0,3,Monsoon,Evening,28.0,,,,,87,25.7,Very_Humid,8.3,,,162,11.7
22	Agartala,Tripura,23.8315,91.2868,2022-08-05 20:00:00,2022,8,5,20,4,Friday,31,0,3,Monsoon,Evening,27.9,,,,,85,25.2,Humid,9.7,,,158,16.2
23	Agartala,Tripura,23.8315,91.2868,2022-08-05 21:00:00,2022,8,5,21,4,Friday,31,0,3,Monsoon,Night_Late,27.7,,,,,88,25.4,Very_Humid,10.6,,,14,6.8
24	Agartala,Tripura,23.8315,91.2868,2022-08-05 22:00:00,2022,8,5,22,4,Friday,31,0,3,Monsoon,Night_Late,27.5,,,,,91,25.9,Very_Humid,10.5,,,14,6.8
25	Agartala,Tripura,23.8315,91.2868,2022-08-05 23:00:00,2022,8,5,23,4,Friday,31,0,3,Monsoon,Night_Late,27.0,,,,,93,25.7,Very_Humid,8.7,,,16,6.8
26	Agartala,Tripura,23.8315,91.2868,2022-08-06 00:00:00,2022,8,6,0,5,Saturday,31,1,3,Monsoon,Night,26.6,,,,,93,25.4,Very_Humid,8.4,,,133,15.6
27	Agartala,Tripura,23.8315,91.2868,2022-08-06 01:00:00,2022,8,6,1,5,Saturday,31,1,3,Monsoon,Night,26.0,,,,,96,25.2,Very_Humid,7.3,,,123,15.6
28	Agartala,Tripura,23.8315,91.2868,2022-08-06 02:00:00,2022,8,6,2,5,Saturday,31,1,3,Monsoon,Night,25.9,,,,,96,25.2,Very_Humid,8.4,,,137,15.6
29	Agartala,Tripura,23.8315,91.2868,2022-08-06 03:00:00,2022,8,6,3,5,Saturday,31,1,3,Monsoon,Night,25.8,,,,,96,25.1,Very_Humid,6.9,,,96,15.6
30	Agartala,Tripura,23.8315,91.2868,2022-08-06 04:00:00,2022,8,6,4,5,Saturday,31,1,3,Monsoon,Night,25.7,,,,,97,25.1,Very_Humid,4.5,,,61,13.6
31	Agartala,Tripura,23.8315,91.2868,2022-08-06 05:00:00,2022,8,6,5,5,Saturday,31,1,3,Monsoon,Night,25.7,,,,,96,24.9,Very_Humid,7.2,,,87,15.6
32	Agartala,Tripura,23.8315,91.2868,2022-08-06 06:00:00,2022,8,6,6,5,Saturday,31,1,3,Monsoon,Early_Morning,26.4,,,,,93,25.3,Very_Humid,5.2,,,13,6.8
33	Agartala,Tripura,23.8315,91.2868,2022-08-06 07:00:00,2022,8,6,7,5,Saturday,31,1,3,Monsoon,Early_Morning,27.4,,,,,91,25.7,Very_Humid,5.1,,,13,6.8
34	Agartala,Tripura,23.8315,91.2868,2022-08-06 08:00:00,2022,8,6,8,5,Saturday,31,1,3,Monsoon,Early_Morning,27.9,,,,,87,25.6,Very_Humid,7.3,,,13,6.8
35	Agartala,Tripura,23.8315,91.2868,2022-08-06 09:00:00,2022,8,6,9,5,Saturday,31,1,3,Monsoon,Morning,28.2,,,,,86,25.7,Very_Humid,6.1,,,87,15.6
36	Agartala,Tripura,23.8315,91.2868,2022-08-06 10:00:00,2022,8,6,10,5,Saturday,31,1,3,Monsoon,Morning,29.1,,,,,81,25.5,Humid,5.0,,,69,16.6
37	Agartala,Tripura,23.8315,91.2868,2022-08-06 11:00:00,2022,8,6,11,5,Saturday,31,1,3,Monsoon,Morning,30.2,,,,,76,25.5,Humid,4.7,,,58,16.2

38	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	12:00:00, 2022, 8, 6, 12, 5, Saturday, 31, 1, 3, Monsoon, Morning, 31.6, , , , 73, 26.2, Humid, 3.8, , , 73, 17.3,
39	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	13:00:00, 2022, 8, 6, 13, 5, Saturday, 31, 1, 3, Monsoon, Afternoon, 32.3, , , , 70, 26.0, Comfortable, 2.4, , ,
40	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	14:00:00, 2022, 8, 6, 14, 5, Saturday, 31, 1, 3, Monsoon, Afternoon, 32.3, , , , 69, 25.8, Comfortable, 5.9, , ,
41	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	15:00:00, 2022, 8, 6, 15, 5, Saturday, 31, 1, 3, Monsoon, Afternoon, 31.8, , , , 69, 25.4, Comfortable, 8.0, , ,
42	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	16:00:00, 2022, 8, 6, 16, 5, Saturday, 31, 1, 3, Monsoon, Afternoon, 30.8, , , , 72, 25.1, Humid, 4.0, , , 185, 15,
43	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	17:00:00, 2022, 8, 6, 17, 5, Saturday, 31, 1, 3, Monsoon, Afternoon, 29.3, , , , 82, 26.0, Humid, 5.7, , , 198, 15,
44	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	18:00:00, 2022, 8, 6, 18, 5, Saturday, 31, 1, 3, Monsoon, Evening, 28.3, , , , 89, 26.2, Very_Humid, 3.4, , , 144,
45	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	19:00:00, 2022, 8, 6, 19, 5, Saturday, 31, 1, 3, Monsoon, Evening, 28.1, , , , 88, 26.0, Very_Humid, 3.3, , , 103,
46	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	20:00:00, 2022, 8, 6, 20, 5, Saturday, 31, 1, 3, Monsoon, Evening, 27.6, , , , 89, 25.7, Very_Humid, 4.0, , , 117,
47	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	21:00:00, 2022, 8, 6, 21, 5, Saturday, 31, 1, 3, Monsoon, Night_Late, 27.7, , , , 89, 25.7, Very_Humid, 5.6, , ,
48	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	22:00:00, 2022, 8, 6, 22, 5, Saturday, 31, 1, 3, Monsoon, Night_Late, 27.3, , , , 91, 25.8, Very_Humid, 5.4, , ,
49	Agartala, Tripura, 23.8315, 91.2868, 2022-08-06	23:00:00, 2022, 8, 6, 23, 5, Saturday, 31, 1, 3, Monsoon, Night_Late, 26.6, , , , 95, 25.7, Very_Humid, 5.0, , ,
50	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	00:00:00, 2022, 8, 7, 0, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.6, , , , 95, 25.7, Very_Humid, 4.6, , , 135, 7.2,
51	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	01:00:00, 2022, 8, 7, 1, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.3, , , , 95, 25.5, Very_Humid, 3.6, , , 90, 5.4, 1,
52	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	02:00:00, 2022, 8, 7, 2, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.1, , , , 97, 25.5, Very_Humid, 2.8, , , 50, 4.7, 1,
53	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	03:00:00, 2022, 8, 7, 3, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.2, , , , 97, 25.7, Very_Humid, 5.6, , , 50, 8.6, 1,
54	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	04:00:00, 2022, 8, 7, 4, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.2, , , , 97, 25.7, Very_Humid, 5.8, , , 68, 10.4,
55	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	05:00:00, 2022, 8, 7, 5, 6, Sunday, 31, 1, 3, Monsoon, Night, 26.3, , , , 97, 25.8, Very_Humid, 5.8, , , 68, 13.0,
56	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	06:00:00, 2022, 8, 7, 6, 6, Sunday, 31, 1, 3, Monsoon, Early_Morning, 27.9, , , , 92, 26.5, Very_Humid, 5.1, , ,
57	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	07:00:00, 2022, 8, 7, 7, 6, Sunday, 31, 1, 3, Monsoon, Early_Morning, 29.4, , , , 81, 25.9, Humid, 6.3, , , 121, 1,
58	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	08:00:00, 2022, 8, 7, 8, 6, Sunday, 31, 1, 3, Monsoon, Early_Morning, 30.7, , , , 72, 25.1, Humid, 6.4, , , 142, 1,
59	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	09:00:00, 2022, 8, 7, 9, 6, Sunday, 31, 1, 3, Monsoon, Morning, 31.7, , , , 67, 24.8, Comfortable, 6.6, , , 151, 1,
60	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	10:00:00, 2022, 8, 7, 10, 6, Sunday, 31, 1, 3, Monsoon, Morning, 32.5, , , , 65, 25.2, Comfortable, 5.9, , , 166,
61	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	11:00:00, 2022, 8, 7, 11, 6, Sunday, 31, 1, 3, Monsoon, Morning, 33.0, , , , 65, 25.4, Comfortable, 4.2, , , 200,
62	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	12:00:00, 2022, 8, 7, 12, 6, Sunday, 31, 1, 3, Monsoon, Morning, 33.5, , , , 64, 25.8, Comfortable, 3.7, , , 241,
63	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	13:00:00, 2022, 8, 7, 13, 6, Sunday, 31, 1, 3, Monsoon, Afternoon, 33.5, , , , 64, 25.6, Comfortable, 3.7, , , 24,
64	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	14:00:00, 2022, 8, 7, 14, 6, Sunday, 31, 1, 3, Monsoon, Afternoon, 33.7, , , , 63, 25.6, Comfortable, 3.6, , , 22,
65	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	15:00:00, 2022, 8, 7, 15, 6, Sunday, 31, 1, 3, Monsoon, Afternoon, 33.6, , , , 62, 25.4, Comfortable, 3.3, , , 18,
66	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	16:00:00, 2022, 8, 7, 16, 6, Sunday, 31, 1, 3, Monsoon, Afternoon, 32.9, , , , 66, 25.6, Comfortable, 3.1, , , 15,
67	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	17:00:00, 2022, 8, 7, 17, 6, Sunday, 31, 1, 3, Monsoon, Afternoon, 31.2, , , , 70, 25.0, Comfortable, 7.3, , , 22,
68	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	18:00:00, 2022, 8, 7, 18, 6, Sunday, 31, 1, 3, Monsoon, Evening, 29.8, , , , 75, 24.9, Humid, 8.0, , , 188, 14.8, 1,
69	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	19:00:00, 2022, 8, 7, 19, 6, Sunday, 31, 1, 3, Monsoon, Evening, 29.1, , , , 79, 25.1, Humid, 6.9, , , 186, 14.0, 1,
70	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	20:00:00, 2022, 8, 7, 20, 6, Sunday, 31, 1, 3, Monsoon, Evening, 28.8, , , , 81, 25.2, Humid, 10.1, , , 176, 17.6,
71	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	21:00:00, 2022, 8, 7, 21, 6, Sunday, 31, 1, 3, Monsoon, Night_Late, 28.4, , , , 83, 25.2, Humid, 10.8, , , 176, 15,
72	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	22:00:00, 2022, 8, 7, 22, 6, Sunday, 31, 1, 3, Monsoon, Night_Late, 27.8, , , , 85, 25.1, Humid, 11.5, , , 182, 26,
73	Agartala, Tripura, 23.8315, 91.2868, 2022-08-07	23:00:00, 2022, 8, 7, 23, 6, Sunday, 31, 1, 3, Monsoon, Night_Late, 27.2, , , , 88, 25.0, Very_Humid, 7.6, , , 17,
74	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	00:00:00, 2022, 8, 8, 0, 0, Monday, 32, 0, 3, Monsoon, Night, 26.6, , , , 94, 25.5, Very_Humid, 6.1, , , 93, 12.6,
75	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	01:00:00, 2022, 8, 8, 1, 0, Monday, 32, 0, 3, Monsoon, Night, 26.5, , , , 95, 25.7, Very_Humid, 5.8, , , 83, 11.9,
76	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	02:00:00, 2022, 8, 8, 2, 0, Monday, 32, 0, 3, Monsoon, Night, 26.3, , , , 96, 25.6, Very_Humid, 4.8, , , 42, 10.4,
77	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	03:00:00, 2022, 8, 8, 3, 0, Monday, 32, 0, 3, Monsoon, Night, 25.9, , , , 97, 25.3, Very_Humid, 9.0, , , 61, 15.5,
78	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	04:00:00, 2022, 8, 8, 4, 0, Monday, 32, 0, 3, Monsoon, Night, 25.7, , , , 96, 25.0, Very_Humid, 9.7, , , 63, 18.7,
79	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	05:00:00, 2022, 8, 8, 5, 0, Monday, 32, 0, 3, Monsoon, Night, 26.1, , , , 96, 25.9, Very_Humid, 10.3, , , 78, 18.4,
80	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	06:00:00, 2022, 8, 8, 6, 0, Monday, 32, 0, 3, Monsoon, Early_Morning, 26.7, , , , 94, 25.6, Very_Humid, 10.0, , ,
81	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	07:00:00, 2022, 8, 8, 7, 0, Monday, 32, 0, 3, Monsoon, Early_Morning, 28.0, , , , 87, 25.7, Very_Humid, 10.6, , ,
82	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	08:00:00, 2022, 8, 8, 8, 0, Monday, 32, 0, 3, Monsoon, Early_Morning, 29.3, , , , 82, 25.9, Humid, 10.9, , , 73, 1,
83	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	09:00:00, 2022, 8, 8, 9, 0, Monday, 32, 0, 3, Monsoon, Morning, 30.3, , , , 77, 25.9, Humid, 11.1, , , 77, 26.3, Fr
84	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	10:00:00, 2022, 8, 8, 10, 0, Monday, 32, 0, 3, Monsoon, Morning, 31.8, , , , 71, 26.0, Humid, 10.5, , , 94, 27.7, F
85	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	11:00:00, 2022, 8, 8, 11, 0, Monday, 32, 0, 3, Monsoon, Morning, 32.7, , , , 68, 26.0, Comfortable, 9.4, , , 113,
86	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	12:00:00, 2022, 8, 8, 12, 0, Monday, 32, 0, 3, Monsoon, Morning, 33.2, , , , 64, 25.5, Comfortable, 8.5, , , 118,
87	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	13:00:00, 2022, 8, 8, 13, 0, Monday, 32, 0, 3, Monsoon, Afternoon, 33.0, , , , 66, 25.9, Comfortable, 8.2, , , 11,
88	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	14:00:00, 2022, 8, 8, 14, 0, Monday, 32, 0, 3, Monsoon, Afternoon, 32.4, , , , 67, 25.6, Comfortable, 9.2, , , 12,
89	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	15:00:00, 2022, 8, 8, 15, 0, Monday, 32, 0, 3, Monsoon, Afternoon, 30.9, , , , 76, 26.2, Humid, 12.3, , , 148, 24,
90	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	16:00:00, 2022, 8, 8, 16, 0, Monday, 32, 0, 3, Monsoon, Afternoon, 30.4, , , , 77, 25.8, Humid, 14.7, , , 158, 26,
91	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	17:00:00, 2022, 8, 8, 17, 0, Monday, 32, 0, 3, Monsoon, Afternoon, 30.2, , , , 93, 24.8, Humid, 12.7, , , 173, 30,
92	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	18:00:00, 2022, 8, 8, 18, 0, Monday, 32, 0, 3, Monsoon, Evening, 27.9, , , , 84, 24.9, Humid, 14.2, , , 164, 31.7,
93	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	19:00:00, 2022, 8, 8, 19, 0, Monday, 32, 0, 3, Monsoon, Evening, 27.0, , , , 91, 25.4, Very_Humid, 13.0, , , 156,
94	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	20:00:00, 2022, 8, 8, 20, 0, Monday, 32, 0, 3, Monsoon, Evening, 26.9, , , , 91, 25.3, Very_Humid, 12.0, , , 141,
95	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	21:00:00, 2022, 8, 8, 21, 0, Monday, 32, 0, 3, Monsoon, Night_Late, 26.8, , , , 90, 25.1, Very_Humid, 13.1, , , 1,
96	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	22:00:00, 2022, 8, 8, 22, 0, Monday, 32, 0, 3, Monsoon, Night_Late, 26.5, , , , 91, 24.9, Very_Humid, 13.7, , , 1,
97	Agartala, Tripura, 23.8315, 91.2868, 2022-08-08	23:00:00, 2022, 8, 8, 23, 0, Monday, 32, 0, 3, Monsoon, Night_Late, 26.3, , , , 91, 24.8, Very_Humid, 12.3, , , 1,
98	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	00:00:00, 2022, 8, 9, 0, 1, Tuesday, 32, 0, 3, Monsoon, Night, 26.1, , , , 93, 24.9, Very_Humid, 9.9, , , 136, 22,
99	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	01:00:00, 2022, 8, 9, 1, 1, Tuesday, 32, 0, 3, Monsoon, Night, 26.1, , , , 93, 24.9, Very_Humid, 9.4, , , 122, 17,
100	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	02:00:00, 2022, 8, 9, 2, 1, Tuesday, 32, 0, 3, Monsoon, Night, 25.9, , , , 93, 24.8, Very_Humid, 8.7, , , 120, 16,
101	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	03:00:00, 2022, 8, 9, 3, 1, Tuesday, 32, 0, 3, Monsoon, Night, 25.8, , , , 93, 24.6, Very_Humid, 8.6, , , 112, 15,
102	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	04:00:00, 2022, 8, 9, 4, 1, Tuesday, 32, 0, 3, Monsoon, Night, 25.6, , , , 94, 24.5, Very_Humid, 9.3, , , 103, 16,
103	Agartala, Tripura, 23.8315, 91.2868, 2022-08-09	05:00:00, 2022, 8, 9, 5, 1, Tuesday, 32, 0, 3, Monsoon, Night, 25.0, , , , 90, 24.3, Very_Humid, 10.0, , , 116, 26,

Implementation

Classes Used and Their Logic

1. User Class

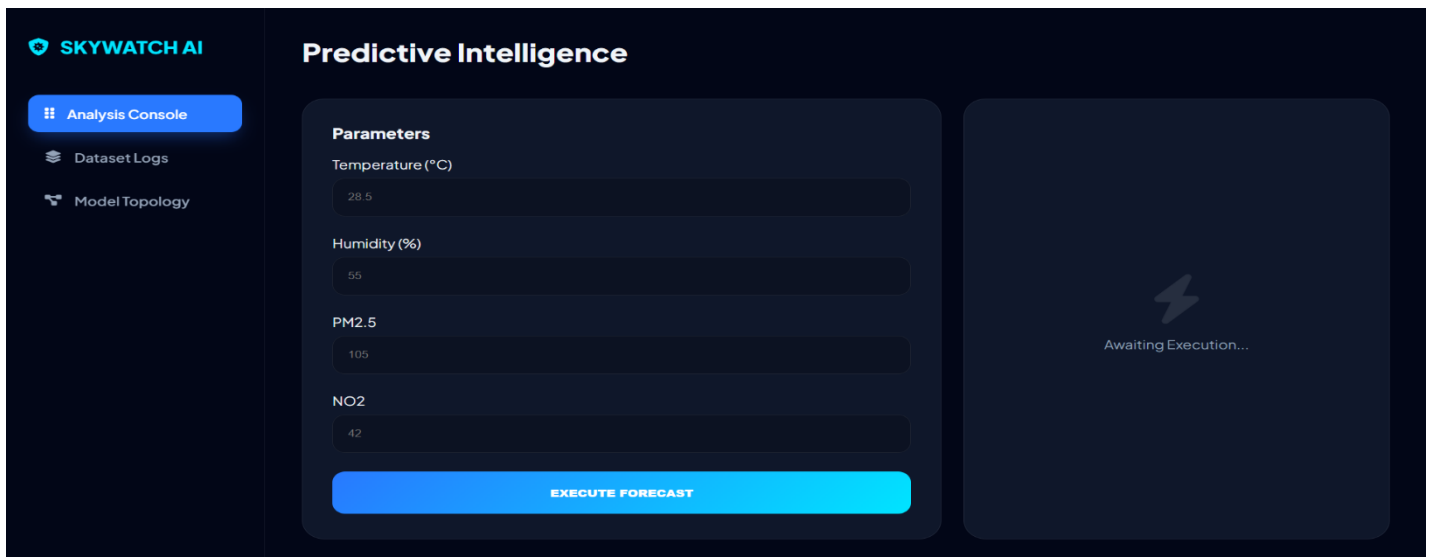
This class is responsible for providing input to the system. The user enters pollution parameters such as PM2.5, PM10, etc. These values are stored in an array format and passed to the next layer for processing. It acts as the starting point of the system.

2. FlaskApp Class

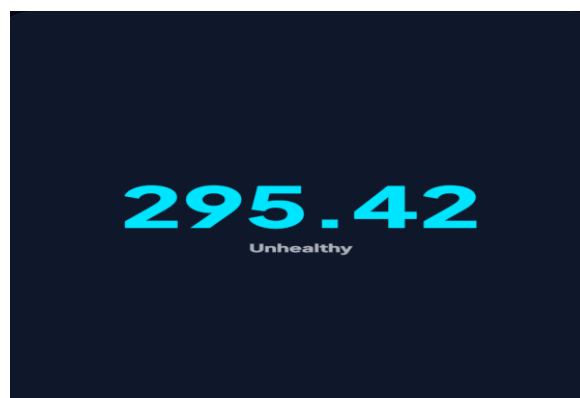
This class handles the communication between the user interface and the backend. It receives the input data from the user through a web form, processes the request, and sends the data to the prediction model. After getting the result, it displays the AQI value and its category on the web page.

3. AQIPredictor Class

This class contains the main logic of prediction. It loads the trained machine learning model and takes the input data array. The model processes this data and predicts the AQI value. It then returns the result to the FlaskApp class.



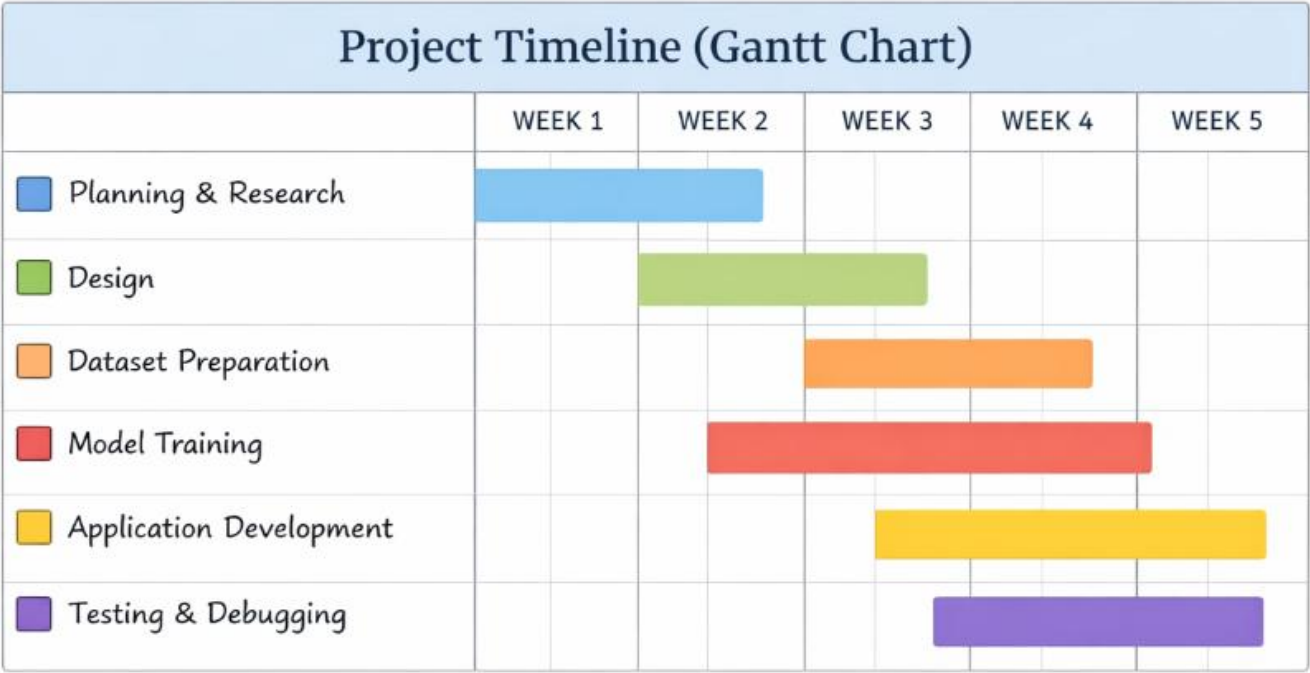
Output:-



Future Enhancements

- **Real-Time AQI Integration:**
The system can be enhanced by integrating real-time air pollution APIs and IoT sensors to provide live AQI prediction and monitoring instead of using only historical datasets.
- **Advanced Machine Learning Models:**
More advanced machine learning and deep learning algorithms such as Random Forest, XGBoost, and Neural Networks can be implemented to improve prediction accuracy and efficiency.
- **Mobile Application Development:**
A mobile application version of the system can be developed so users can easily access AQI predictions and pollution alerts from smartphones anytime and anywhere.
- **Graphical Data Visualization:**
Graphs, charts, and dashboards can be added to visualize pollution trends, AQI variations, and historical data analysis in a more understandable way.
- **Cloud Database Integration:**
The project can be connected with cloud databases to store user data, historical AQI records, and real-time pollution information securely.
- **Location-Based Prediction:**
GPS and location services can be integrated to provide city-wise or area-wise AQI predictions automatically based on the user's location.
- **Alert and Notification System:**
The system can be upgraded to send notifications or warning alerts when AQI levels become unhealthy or hazardous for public health.
- **Multi-Parameter Environmental Monitoring:**
Future versions of the project can include additional environmental factors such as temperature, humidity, weather conditions, and wind speed for more accurate and intelligent AQI prediction.

Gantt Chart



Conclusion

The Air Quality Index (AQI) Prediction System is a machine learning-based application developed to predict air quality levels using pollution parameters such as PM2.5, PM10, and NO₂. The project successfully combines data preprocessing, machine learning techniques, and web development to create an efficient and user-friendly AQI prediction system.

The system analyzes pollution data, trains a machine learning model, and provides instant AQI predictions through a Flask web application. It also classifies AQI values into different categories such as Good, Moderate, Poor, and Unhealthy, helping users understand air quality conditions easily. The use of machine learning improves prediction accuracy and enables faster analysis compared to traditional monitoring methods.

This project helps increase awareness about environmental pollution and its impact on human health. It demonstrates how modern technologies like Python, Scikit-learn, Flask, and data analysis techniques can be used to solve real-world environmental problems effectively.

Overall, the project provides a simple, cost-effective, and intelligent solution for AQI prediction and environmental monitoring. In the future, the system can be further enhanced with real-time data integration, IoT sensors, advanced machine learning algorithms, graphical dashboards, and mobile application support for better accuracy and wider usability.

References

Book References

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. California: O'Reilly Media, 2019.
- [2] T. Mitchell, *Machine Learning*, 1st ed. New York: McGraw-Hill, 1997.

Website References

- [3] Scikit-learn, "Machine Learning in Python," Scikit-learn. [Online]. Available: <https://scikit-learn.org>
- [4] Flask, "Flask Documentation," Pallets Projects. [Online]. Available: <https://flask.palletsprojects.com>

Conference Paper

- [5] R. Singh and P. Verma, "Air Quality Prediction using Machine Learning," in Proc. IEEE Int. Conf. on Environmental Informatics, 2021, pp. 85-90.

Journal Article

- [6] S. Gupta and M. Sharma, "Air Pollution Prediction using ML Techniques," International Journal of Environmental Science, vol. 6, no. 3, pp. 112-118, 2022.